Sandboxing, Binder, boot, ART, Partition etc

Java vs Android: Compilation Process Comparison

This diagram illustrates the fundamental difference between how standard Java applications and Android applications are compiled and executed.

Standard Java Compilation (Left Side)

1. **Java Source Code** → Written by developers in `.java` files
2. **Java Compiler** → Compiles the source code
3. **Java Byte Code** → Produces platform-independent `.class` files
4. **Java VM (Virtual Machine)** → Executes the bytecode on any platform with a JVM

This is the traditional Java approach where bytecode runs on the Java Virtual Machine, making it "write once, run anywhere."

Android Compilation Process (Right Side)

1. **Java Source Code** → Developers write Android apps in Java (or Kotlin)
2. **Java Compiler** → Compiles source code into standard Java bytecode
3. **Java Byte Code** → Standard `.class` files are generated
4. **Dex Compiler** → Converts Java bytecode into Dalvik bytecode (`.dex` files)
5. **Dalvik Byte Code** → Optimized bytecode specifically designed for Android
6. **Dalvik VM** → Executes the Dalvik bytecode on Android devices

Key Differences

Why the extra step?

Android uses the **Dex (Dalvik Executable) compiler** to convert standard Java bytecode into a format optimized for mobile devices. The Dalvik VM was specifically designed for Android to:

- **Reduce memory footprint** – Mobile devices have limited RAM
- **Optimize for battery efficiency** – Power consumption is critical on mobile
- **Enable multiple instances** – Each app runs in its own VM instance
- **Improve performance** – Register-based architecture (Dalvik) vs stack-based (JVM)

Modern Update: Starting from Android 5.0 (Lollipop), Android replace Dalvik VM with **ART (Android Runtime)**, which uses ahead-of-time (AOT) compilation instead of just-in-time (JIT) compilation for even better performance.

Understanding Dalvik Byte Code

Dalvik byte code is a specially optimized bytecode format designed specifically for resource-constrained mobile environments with limited memory and processing power.

Key Differences: JVM vs Dalvik Byte Code

JVM Byte Code Structure

- Produces **multiple `.class` files** – one for each Java class in your application
- If your app has 50 Java files, you'll get 50 separate `.class` files
- Results in more files to manage and load

Dalvik Byte Code Structure

- Consolidates everything into a **single `.dex` (Dalvik Executable) file**
- Regardless of how many Java/Kotlin classes your app contains, all bytecode is packaged into one `.dex` file
- More efficient for mobile devices with limited resources

Why This Matters

The consolidation into a single `.dex` file offers several advantages for Android devices:

1. **Reduced Memory Footprint** – One file is easier to load and manage than dozens or hundreds of `.class` files
2. **Faster App Loading** – Less file I/O operations mean quicker startup times
3. **Optimized Storage** – Better compression and removal of redundant information across classes
4. **Efficient Execution** – Streamlined format designed for mobile processors and limited RAM

Dalvik byte code represents Google's smart optimization for mobile platforms. By converting multiple `.class` files into a single, optimized `.dex` file, Android ensures apps run smoothly even on devices with modest hardware specifications. This architecture was a key innovation that helped Android scale across billions of devices worldwide, from budget smartphones to flagship models.

Modern Note: While newer Android versions (5.0+) use ART (Android Runtime) instead of Dalvik VM, the `.dex` format remains central to Android's compilation process, continuing to provide these optimization benefits.

Android Application Sandboxing

Application sandboxing is one of Android's fundamental security mechanisms that ensures apps cannot interfere with each other or access unauthorized data. Let's understand how this works.

How Android Sandboxing Works

Android leverages the **Linux user-based protection model** to isolate applications from one another. Here's the process:

Linux User ID (UID) Assignment

- In Linux systems, each user receives a unique **User ID (UID)**
- Users are segregated so that one user cannot access another user's data
- All resources belonging to a particular user run with the same privileges

Android's Implementation

- Each Android application is assigned its own **unique Linux UID** at installation
- Every app runs as a **separate process** with its own dedicated user ID
- This sandboxing occurs at the **kernel level**, making it extremely robust

Understanding the Diagram

The diagram shows two separate app sandboxes running on an Android device:

Left Sandbox (App 1):

- **Linux User ID: 12345**
- The application runs with this unique ID
- All resources (Files, Database, SMS, Logs, Network, Sensors) are owned by UID 12345
- No other app can access these resources without explicit permission

Right Sandbox (App 2):

- **Linux User ID: 54321**
- Completely isolated from App 1
- Has its own separate resources with UID 54321
- Cannot access App 1's data or resources

The dashed arrow between sandboxes represents that apps are isolated—they cannot directly access each other's resources unless explicitly permitted through Android's permission system.

Key Security Benefits

1. **Process Isolation** – Each app runs in its own process space, preventing interference
2. **Resource Protection** – Files, databases, and other resources are protected by Linux file permissions
3. **Kernel-Level Security** – Sandboxing is enforced at the operating system kernel level, not just the application layer
4. **Universal Application** – Applies to **all apps** including:
    - Third-party applications from Play Store
    - Pre-installed system applications
    - Native Android OS applications

Real-World Implications

This sandboxing architecture means:

- **A malicious app cannot steal data** from your banking app
- **Games cannot access your messages** without permission
- **Photo editing apps cannot read your location history** unless you explicitly allow it
- **Even if one app is compromised**, it remains contained within its sandbox

Exceptions and Permissions

While sandboxing provides strong isolation by default, Android apps can request **permissions** to access:

- Shared resources (camera, microphone, location)
- User data (contacts, photos, messages)
- Other apps' data (through Content Providers with proper permissions)

These permissions must be explicitly granted by the user, adding an additional layer of security on top of the sandbox model.

1️⃣ What is Binder in Android?

In Android, **Binder** is the primary IPC (Inter-Process Communication) mechanism.
It allows different processes (apps, system services) to talk to each other securely and efficiently.

2️⃣ Who is the Context Manager?

The **Context Manager** is a special Binder service.

Think of it like: A central directory that maps **service names → service handles (tokens).** It works like a **name service**.

3️⃣ Step-by-Step Working Process

🔹 Step 1: Service Registration

- Every system service (called a **Binder service**) must register itself with the **context manager**.
- During registration:
  - It provides its **service name**
  - It gets assigned a **Binder token**

🔹 Step 2: Token Assignment

- Each Binder service gets a:
  - **32-bit unique token**
- This token:
  - Is unique across the system
  - Identifies the service internally
  - Acts like a communication handle

Think of it as

```
Service Name  →  Token (32-bit value) →  Used for IPC
```

🔹 Step 3: Client Lookup

When a client wants to use a service:

1. Client knows only the **service name**
2. It sends a request to the **context manager**
3. Context manager:
   - Resolves the name
   - Returns the corresponding **token**
4. Client now uses this token to communicate directly with the service

4️⃣ Why Context Manager is Important

Without it:

- Clients would need to know process IDs
- Service discovery would be complex

- Security control would be difficult

With it:

- Loose coupling between services and clients
- Centralized service lookup
- Secure handle distribution

Complete Flow

1. Service registers with Context Manager
2. Service receives unique 32-bit Binder token
3. Client requests service using service name
4. Context Manager resolves name → returns token
5. Client uses token to communicate with service

Android Boot Process

When an Android device is powered on, it follows a defined sequence of steps to load the operating system, initialize hardware, and start system services. The boot process ensures secure startup, memory initialization, and preparation of the user environment.

1. Boot ROM

The Boot ROM is the first code executed when the device is powered on.

- It is stored in read-only memory (ROM) inside the chipset.
- It initializes basic hardware components such as CPU, memory controller, and device hardware.
- It scans the boot media to locate the bootloader.
- Once detected, the bootloader code is copied into RAM.
- Execution control is then transferred to the bootloader.

The Boot ROM cannot be modified and ensures the authenticity of the boot chain, forming the foundation of secure boot.

2. Boot Loader

The Boot Loader is a program executed before the operating system begins functioning. It prepares the environment required for loading the OS.

It operates in two stages:

a) Initial Program Load (IPL)

- Performs hardware detection and setup.
- Initializes basic memory configuration.
- Locates the Second Program Load (SPL).

b) Second Program Load (SPL)

- Copied into RAM for execution.
- Loads the Linux Kernel into memory.
- Supports different boot modes such as:
    - Fastboot mode
    - Recovery mode
    - Normal boot mode
- Provides access to boot media and file system.

After completing its operations, the bootloader transfers control to the Linux Kernel.

3. Linux Kernel

The Linux Kernel is the core of the Android operating system.

Its responsibilities include:

- Process management
- Memory management
- Device driver initialization
- Security enforcement
- Hardware abstraction

Once loaded:

- The kernel initializes memory management units and CPU caches.
- Virtual memory support is enabled.
- The root file system (rootfs) is mounted.
- The kernel searches for the `init` process in rootfs.
- The init process is launched as the first user-space process.

At this stage, the system transitions from kernel space to user space.

4. Init Process

The `init` process is the first user-space process started by the kernel.

Key characteristics:

- It has Process ID (PID) 1.
- It is the parent of all other processes.
- It parses configuration scripts such as `init.rc`.

Functions of Init:

- Mounts essential file systems.
- Configures system properties.
- Starts system daemons and services.
- Initializes device-specific services.

During this stage, the Android logo appears on the screen, indicating system-level initialization is in progress.

5. Zygote and Dalvik/ART

After init completes early setup, it starts the Zygote process.

Zygote plays a critical role in Android application management.

Functions of Zygote:

- Initializes the Dalvik Virtual Machine (in older Android versions) or ART (Android Runtime in modern versions).
- Loads core Java libraries.
- Preloads shared classes and resources.
- Creates new application processes using a fork mechanism.

Because applications are forked from Zygote:

- Memory consumption is optimized.
- Shared code is reused.
- Process creation becomes faster.

Zygote improves system efficiency and performance by sharing common runtime components across applications.

6. System Server

The System Server is started by Zygote and is responsible for launching Android's core system services.

It starts major services such as:

- Power Manager
- Activity Manager
- Telephony Registry
- Package Manager
- Context Manager (Binder service manager)

Functions of System Server:

- Controls application lifecycle.
- Manages system resources.
- Handles telephony and network services.
- Enforces security and permission management.
- Registers services with Binder for IPC communication.

Once the System Server completes initialization, the Android framework becomes fully operational and the home screen is displayed.

Complete Boot Flow Summary

Power On
→ Boot ROM executes
→ Bootloader (IPL → SPL)
→ Linux Kernel loads
→ Root file system mounted
→ Init process starts
→ Zygote initializes runtime
→ System Server launches core services
→ Android system ready for user interaction

Security Perspective

The Android boot process follows a chained execution model:

- Each stage verifies the next stage.
- Prevents unauthorized code execution.
- Ensures system integrity.
- Forms the base for Secure Boot implementation.

Android Runtime (ART)

1. Introduction

Android Runtime (ART) is the managed runtime environment used by the Android operating system to execute applications. It replaced the Dalvik Virtual Machine starting from Android 5.0 (Lollipop).

ART is responsible for:

- Executing application bytecode
- Managing memory
- Handling garbage collection
- Providing core Java libraries

It ensures that Android applications run efficiently, securely, and consistently across devices.

2. Evolution: Dalvik vs ART

Earlier versions of Android used **Dalvik Virtual Machine (DVM)**.

Dalvik

- Used Just-In-Time (JIT) compilation
- Converted bytecode into machine code during runtime
- Slower startup time
- Higher CPU usage during execution

ART

- Uses Ahead-Of-Time (AOT) compilation
- Compiles bytecode into native machine code during installation
- Faster app launch
- Better performance
- Improved battery efficiency

Modern Android versions use a hybrid model combining AOT and JIT for optimization.

3. How Android Runtime Works

When an Android app is developed:

- Source code is written in Java or Kotlin
- It is compiled into bytecode
- Converted into .dex (Dalvik Executable) format

During installation:

- ART compiles the .dex file into native machine code
- This native code is stored on the device

When the user launches the app:

- The precompiled native code is executed directly
- No need for runtime translation

This reduces execution overhead.

4. Core Components of Android Runtime

1. Core Libraries

- Provide Java API support
- Include data structures, utilities, networking, I/O operations
- Enable developers to use standard Java classes

2. Execution Engine

- Executes compiled native code
- Supports both AOT and JIT compilation
- Optimizes frequently used code paths

3. Garbage Collection (GC)

- Automatically manages memory
- Reclaims unused objects
- Prevents memory leaks
- Reduces OutOfMemory errors

ART includes improved garbage collection compared to Dalvik:

- Lower pause times
- More efficient heap management

5. Role of Zygote in Android Runtime

During boot:

- The Zygote process starts first
- It initializes the Android Runtime
- Loads core libraries
- Preloads common classes

When a new app is launched:

- The app process is forked from Zygote
- It inherits preloaded runtime components

This improves:

- Startup speed
- Memory efficiency

6. Advantages of Android Runtime (ART)

1. Faster application startup
2. Improved performance
3. Better battery efficiency
4. Reduced CPU overhead
5. Improved garbage collection
6. Better debugging and profiling support

7. Security Perspective

Android Runtime enhances security by:

- Running apps in isolated processes
- Enforcing application sandboxing
- Performing runtime verification
- Supporting SELinux enforcement (through kernel integration)

Each app runs in its own instance of ART within its own process, maintaining isolation.

Emulator

- **Definition:**
  An emulator **imitates both hardware + software** of a real device.
- **Key Point:**
  It creates a **virtual environment identical to a real device**.

- **Example:**
  Android Emulator, Genymotion
- **Use:**
  - Run apps as if on a real phone
  - Test hardware features (camera, GPS, etc.)

Simulator

- **Definition:**
  A simulator **only mimics software behavior**, not hardware.
- **Key Point:**
  It does **not replicate real device hardware**
- **Example:**
  iOS Simulator
- **Use:**
  - Test app logic/UI
  - Faster but less realistic

Difference

Feature	Emulator	Simulator
Hardware support	Yes (virtual hardware)	No
Accuracy	High	Moderate
Speed	Slower	Faster
Real environment	Yes	No
Testing capability	Full (hardware + software)	Limited (software only)

1. Finding Attack Surface using Drozer

Command:

dz> run app.package.attacksurface jakhar.aseem.diva

What it does:

- Shows **entry points (attack surface)** of the app

Output includes:

- Activities
- Services
- Broadcast Receivers
- Content Providers

👉 In DIVA:

- 3 Activities
- 1 Content Provider

📌 Meaning:
The **Content Provider** is the main target for **SQL Injection attack**

2. Finding Content Provider URIs

Command:

dz> run app.provider.finduri jakhar.aseem.diva

Output:

content://jakhar.aseem.diva.provider.notesprovider/notes/

content://jakhar.aseem.diva.provider.notesprovider

content://jakhar.aseem.diva.provider.notesprovider/

content://jakhar.aseem.diva.provider.notesprovider/notes

📌 These are **entry points to database queries**

3. Querying Content Provider

Command:

dz> run app.provider.query content://.../notes/

👉 If data returns → **provider is accessible**
👉 If error → might be restricted or wrong URI

📌 As per slide:
Some URIs fail, some succeed → attacker tries all

4. Problem (Manual Testing Issue)

- Too many URIs
- Time-consuming to test manually
- Need automation

5. Injection Scanner (Automated Detection)

Command:

dz> run scanner.provider.injection -a jakhar.aseem.diva

Output:

- Not vulnerable → some cases
- Vulnerable in:
    - **Projection**
    - **Selection**

📌 This tells:

> The app is vulnerable to SQL injection in specific query parameters

6. Core Concept: Projection vs Selection

This is the **MOST IMPORTANT THEORY QUESTION**

✔️ SQL Query Example:

SELECT a, b, c FROM foobar WHERE x=3;

Part	Meaning
`a, b, c`	Projection
`WHERE x=3`	Selection

🔸 Projection (Columns)

- Defines **what data to return**
- Columns or expressions

👉 Example:

SELECT username, password FROM users;

🔸 Selection (Rows)

- Defines **which rows to return**

👉 Example: SELECT * FROM users WHERE id=1;

7. Injection Types (Main Topic)

1. Injection in Selection

What happens:

- Attacker injects into **WHERE clause**

Detection:

dz> run app.provider.query content://.../notes/ -- selection """

👉 Output: Error occurs → vulnerable

📌 Reason: ' breaks SQL syntax

Exploitation Example:

-- selection "* FROM SQLITE_MASTER WHERE type='table'; --"

👉 Result: Lists all tables in database

Next Step:

-- selection "* FROM notes; --"

👉 Result: Extracts full table data

✔️ 2. Injection in Projection

What happens:

- Attacker injects into **SELECT part**

Detection:

dz> run app.provider.query content://.../notes/ -- projection """

👉 Error → vulnerable

Exploitation Example:

-- projection "* FROM SQLITE_MASTER WHERE type='table'; --"

👉 Same effect: Gets database structure

8. Why ‘ (Single Quote) is Used?

👉 It is used to:

- Break SQL syntax
- Trigger error
- Confirm injection

Example: SELECT * FROM notes WHERE name = ''

If app crashes → vulnerable

9. Complete Attack Flow

Step-by-step:

1. Identify package

app.package.list

2. Check attack surface: app.package.attacksurface
3. Find URIs: app.provider.finduri
4. Test query: app.provider.query
5. Run injection scanner: scanner.provider.injection
6. Test manually with '
7. Exploit with SQL payload
8. Extract database

Important Partitions

🔸 1. Bootloader

What it does:

- First program that runs when phone starts
- Initializes hardware
- Decides how to boot

Memory Trick:

👉 **Bootloader = Security Guard at the gate**

Key Points:

- Starts Android kernel
- Can boot into:
    - Normal mode
    - Recovery mode
    - Download mode

🔸 2. Boot Partition (⚙️ Engine)

What it contains:

- Kernel
- RAM disk

Function:

- Starts the OS

Memory Trick:

👉 **Boot = Engine of the phone**

📌 Without this → phone cannot start

🔸 3. Recovery Partition

What it does:

- Allows system repair and maintenance

Functions:

- Factory reset
- Install updates
- Debugging

Memory Trick:

👉 **Recovery = Hospital of phone**

🔸 4. System Partition (🏗️ OS Room)

What it contains:

- Android OS
- Framework
- Libraries
- Pre-installed apps

Key Point:

- Without this → no Android OS

Memory Trick:

👉 **System = Brain of phone**

🔸 5. Userdata Partition

What it contains:

- App data
- User files
- Messages, logs

VERY IMPORTANT FOR EXAM:

👉 This is where **forensic evidence is stored**

🔸 6. Cache Partition

What it stores:

- Frequently used data
- Logs
- Update files

Memory Trick:

👉 **Cache = Short-term memory**

🔸 7. Radio Partition

What it does:

- Handles network communication

Includes:

- Baseband firmware

Memory Trick:

👉 **Radio = SIM/network brain**

4. Forensics View

Where is evidence stored?

✔️ Answer: Mostly in **Userdata partition**

Because it contains:

- App data
- Messages
- Files
- Logs

> “The userdata partition is the primary source of digital evidence in Android forensics.”

5. Commands to View Partitions

(These are practical + viva questions)

✔️ View partitions:

cat /proc/partitions

✔️ View mounted partitions:

cat /proc/mounts

✔️ Disk usage:

df

👉 Shows:

- Total size
- Used space
- Free space

🔷 6. Mapping Partition Paths

👉 Important concept:
Each partition has a **name + actual file path**

Commands:

ls -al /dev/block/platform/.../by-name

OR

cat /proc/emmc

cat /proc/dumchar_info

Why important?

- Helps forensic analyst locate exact storage location

7. BusyBox (Small but Important)

👉 It is a tool that provides **Linux utilities in one package**

Example: busybox fdisk -l

📌 Used for:

- Partition analysis
- Disk inspection

Here is a clear and complete explanation of each numbered component in your Android Application Framework diagram.

1. AndroidManifest.xml (Manifests folder)
   This is the most important configuration file of an Android app. It defines how the app behaves at the system level. It contains information such as app package name, permissions required like internet or camera access, components like activities, services, broadcast receivers, app entry point (main activity), intent filters for linking and communication. Without this file, Android cannot run the app because it acts like the app's identity and rulebook.
2. res (Resources folder)
   This folder stores all non-code resources used in the app. These are things that define the UI and design rather than logic. It includes images (drawable), UI layouts (layout), app icons (mipmap), strings, colors, styles (values). These resources are separated from code so that apps are easier to maintain, support multiple languages, and adapt to different screen sizes.
3. java (Source Code folder)
   This contains the actual programming logic of the application. All Java or Kotlin classes are stored here. It includes main activity classes, business logic, backend processing, event handling, and interactions with the UI. You will also see test folders like androidTest and test, which are used for writing test cases for the app.
4. layout (inside res)
   This folder specifically contains XML files that define the structure of the user interface. Each XML file represents a screen or part of a screen. It defines UI components like buttons, text fields, images, and how they are arranged. The logic for these layouts is handled in the java folder, but the design is defined here.
5. values (inside res)
   This folder stores constant values used across the app. It includes strings.xml for all text used in the app, colors.xml for color definitions, dimens.xml for spacing and sizes, styles.xml for themes and UI styling. This helps in reusability and makes it easier to update content without touching code.
6. Gradle Scripts
   This section manages how the app is built and compiled. It includes multiple files.

build.gradle (Project level)
Defines global configurations like repositories and dependencies used across all modules.

build.gradle (App level)
Defines app-specific configurations such as SDK version, dependencies (libraries), version code, build types (debug/release).

Gradle-wrapper.properties: Defines which Gradle version is used.

proguard-rules.pro: sed for code shrinking and obfuscation to make the app secure and smaller.

Settings.gradle: Defines which modules are included in the project.

Local.properties: Stores local system paths like Android SDK location.

MobSF

Mobile Security Framework (MobSF) in Android Application Pentesting

MobSF (Mobile Security Framework) is an **automated mobile application security testing framework** used to perform **static analysis, dynamic analysis, and malware analysis** of Android, iOS, and Windows mobile applications. It is widely used by **penetration testers, malware analysts, digital forensics investigators, and security researchers** to identify vulnerabilities, insecure coding practices, and privacy leaks in mobile applications.

MobSF simplifies mobile security testing by providing a **web-based interface that automates most security checks** required during a mobile penetration test.

1. Purpose of MobSF in Android Pentesting

In **Android application penetration testing**, MobSF helps analysts to:

- Detect **security vulnerabilities**
- Identify **insecure permissions**
- Find **hardcoded secrets**
- Analyze **source code and APK structure**
- Detect **malware behavior**
- Perform **runtime monitoring**
- Analyze **network traffic**
- Identify **privacy leaks**

MobSF is capable of analyzing:

- **APK files**
- **Android source code**
- **Mobile malware samples**
- **Runtime behavior of apps**

2. Architecture of MobSF

MobSF is mainly divided into **three major analysis engines**:

1. **Static Analysis Engine**
2. **Dynamic Analysis Engine**
3. **Malware Analysis Engine**

It also contains several **supporting modules**:

- API Analyzer
- Certificate Analyzer
- Binary Analyzer
- Manifest Analyzer
- Permission Analyzer
- Tracker Detector
- Network Security Analyzer

3. Components of MobSF

MobSF architecture contains the following core components:

1. Web Interface

MobSF provides a **browser-based dashboard** where users can upload APK files and view results.

Functions include:

- Upload mobile apps
- Start static analysis
- Launch dynamic analysis
- Monitor runtime logs
- Generate reports

2. Static Analysis Engine

Static analysis examines the **application without executing it**.

Steps performed by MobSF:

1. APK extraction
2. Manifest analysis
3. Permission analysis
4. Source code decompilation
5. API analysis
6. Hardcoded secret detection
7. Security misconfiguration detection

Tools internally used:

- apktool
- jadx
- dex2jar
- Java decompilers

4. Static Analysis Modules

4.1 APK Decompiler

MobSF decompiles the APK into readable components. APK structure contains:

classes.dex
AndroidManifest.xml
resources.arsc
res/
assets/
META-INF/

MobSF converts:

- **DEX → Java code**
- **resources → readable format**

4.2 Manifest Analyzer

Analyzes **AndroidManifest.xml**.

Checks include:

- Exported activities
- Broadcast receivers
- Services
- Content providers
- Permissions used
- Debug mode enabled
- Backup enabled

Example vulnerabilities detected:

- `android:debuggable="true"`
- `android:allowBackup="true"`

These can allow attackers to **extract application data**.

4.3 Permission Analyzer

MobSF identifies **dangerous permissions** used by the application.

Examples:

Permission	Risk
READ_SMS	SMS interception
READ_CONTACTS	Privacy leakage
RECORD_AUDIO	Spyware behavior
ACCESS_FINE_LOCATION	Location tracking

MobSF categorizes permissions into:

- Normal
- Dangerous
- Signature
- Special permissions

4.4 API Analyzer

MobSF scans code for **sensitive APIs**. Examples:

```
getDeviceId()
getSubscriberId()
sendTextMessage()
exec()
loadLibrary()
```

Risks:

- Device fingerprinting
- SMS fraud
- Command execution
- Native code exploitation

4.5 Hardcoded Secrets Detection

MobSF detects hardcoded data such as:

- API keys
- Passwords
- Encryption keys
- Tokens
- Private URLs

Example detection: String apiKey = "ABCD12345SECRET";

4.6 Code Analysis

MobSF scans for insecure coding practices such as:

- Weak cryptography
- Insecure random numbers
- Logging sensitive data
- Insecure SSL configuration

Example: TrustManager[] trustAllCerts. This indicates **SSL certificate validation bypass**.

4.7 Certificate Analysis

MobSF checks the APK signing certificate. Information extracted:

- Certificate issuer
- Validity period
- Signature algorithm
- Certificate fingerprint

Weak algorithms like **MD5** or **SHA1** are flagged.

4.8 Tracker Detection

MobSF detects third-party **tracking libraries**.

Examples:

- Google Analytics
- Facebook SDK
- Firebase Analytics

These trackers may collect:

- User behavior
- Device identifiers
- Location data

5. Dynamic Analysis Engine

Dynamic analysis means **analyzing the application while it is running**. MobSF runs the APK inside:

- Android emulator
- Virtual device

Then monitors:

- Runtime behavior
- Network communication
- API calls
- File operations

5.1 Dynamic Analysis Environment

MobSF creates a controlled environment with:

- Rooted Android emulator
- SSL interception
- Runtime instrumentation
- Network monitoring

5.2 Runtime Monitoring

MobSF monitors runtime activity including:

- File system access
- Clipboard access
- Device info collection
- SMS usage
- Camera usage
- Microphone access

5.3 API Call Monitoring

MobSF intercepts runtime API calls such as:

Runtime.exec()
WebView.loadUrl()
sendTextMessage()

These help detect malicious behavior.

5.4 Network Traffic Analysis

MobSF captures network traffic.

It detects:

- HTTP requests
- HTTPS traffic
- API calls
- Data leaks

MobSF uses **proxy interception** similar to tools like **Burp Suite**.

Detected issues include:

- Unencrypted traffic
- Sensitive data in requests
- Weak SSL configuration

5.5 WebView Analysis

Many Android apps use **WebView**. MobSF detects vulnerabilities like:

- JavaScript enabled unnecessarily
- Insecure URL loading
- File access enabled

Example vulnerability: webView.getSettings().setJavaScriptEnabled(true);

6. Malware Analysis Engine

MobSF also performs **Android malware analysis**.

It identifies:

- Trojan behavior
- Spyware activity
- Data exfiltration
- Command and Control communication

Methods used:

- Static signature detection
- Behavioral analysis
- Suspicious API usage

7. Binary Analysis

MobSF analyzes **native libraries (.so files)**.

These are written in:

- C
- C++

Binary analysis detects:

- Buffer overflow risks
- Unsafe native functions
- Obfuscation
- Packed malware

8. Privacy Analysis

MobSF checks if the application collects:

- Contacts
- SMS
- Location
- Call logs
- Device identifiers

It flags apps that **collect excessive user data**.

9. Security Score

MobSF provides a **security score** based on vulnerabilities found.

Example evaluation:

Category	Score
Permissions	Risk
Code Security	Moderate
Network Security	Critical
Data Storage	Moderate

Final score helps determine **overall application security posture**.

10. MobSF Report Generation

MobSF generates a **detailed penetration testing report** including:

- Vulnerability summary
- Detailed findings
- CVSS risk level
- Code snippets
- Recommended remediation

Report formats:

- HTML
- JSON
- PDF

11. Supported File Types

MobSF can analyze:

Android:

- APK
- AAB
- ZIP source code

iOS:

- IPA
- iOS source code

Windows mobile: APPX

12. MobSF Workflow in Android Pentesting

Typical workflow:

1. Upload APK
2. Static analysis performed
3. Review vulnerabilities

4. Start dynamic analysis
5. Install app in emulator
6. Monitor runtime behavior
7. Capture network traffic
8. Generate security report

13. Advantages of MobSF

Advantages include:

- Automated mobile pentesting
- Open source
- Supports multiple platforms
- Easy web interface
- Combines static and dynamic analysis
- Detailed vulnerability reports

14. Limitations of MobSF

Limitations include:

- Cannot detect **all runtime vulnerabilities**
- Some apps detect emulator environment
- Heavy obfuscation may limit static analysis
- Dynamic analysis requires proper emulator setup

15. MobSF vs Other Mobile Pentesting Tools

Tool	Purpose
Mobile Security Framework (MobSF)	Automated analysis
Frida	Runtime instrumentation
Drozer	Exploitation framework
Burp Suite	Network interception

MobSF is often used **alongside these tools** in professional pentesting.

Qark, Sig, Xposed, Debugging

1. Android App Signing Schemes (v1, v2, v3, v4)

Android uses digital signatures to verify that an app is authentic and has not been modified.

v1 Signature Scheme
This is the oldest method, based on JAR signing. It signs only specific parts of the APK (ZIP entries), not the entire file. Because of this, some parts of the file can be changed without affecting the signature.

v2 Signature Scheme
Introduced in Android 7.0. It signs the entire APK file, not just parts. This makes it much more secure because even a small change in the file will break the signature.

v3 Signature Scheme
Introduced in Android 9. It improves on v2 by adding support for key rotation, meaning developers can safely change their signing keys.

v4 Signature Scheme
Introduced in Android 11. It is mainly used to speed up app installation and verification, especially for incremental installs.

2. What is the Janus Vulnerability

The Janus vulnerability (CVE-2017-13156) is a serious Android security flaw that allows attackers to modify an app without changing its signature.

This is dangerous because Android relies on signatures to decide whether an app is trustworthy.

3. Root Cause of the Vulnerability

The vulnerability exists because a file can act as both:

- A valid APK file (Android app package)
- A valid DEX file (compiled Android code)

This dual nature is the core problem.

Why this happens: APK files (ZIP format): They allow extra bytes at the beginning, middle, or between entries. The v1 signature ignores these extra bytes.

DEX files: They allow extra bytes at the end of the file.

Because of this, attackers can insert malicious code into unused parts of the file without affecting the signature.

4. Role of Dalvik/ART Virtual Machine

Android uses Dalvik/ART runtime to execute apps.

How it behaves:

- It checks the file header (magic bytes) to decide file type
- If it detects a DEX header → treats it as a DEX file
- Otherwise → treats it as an APK file

Problem:
The same file can be interpreted differently depending on how it is read. This confusion allows malicious code to be executed.

5. How the Attack Works

Step-by-step:

1. Attacker takes a legitimate APK
2. Injects a malicious DEX file into it
3. Because v1 signature ignores extra bytes, signature remains valid
4. Android verifies the signature and accepts the app as safe
5. Dalvik VM loads the malicious DEX code instead of the original one
6. Malicious code runs on the device
7. Why This is Dangerous

When users update an app:

- Android checks if the signature matches the previous version
- If it matches → permissions are automatically granted

Because the signature is unchanged: The malicious app gets all permissions of the original app

This can lead to:

- Access to sensitive data
- Abuse of high privileges (especially system apps)
- Full device compromise in extreme cases

7. Real Attack Possibilities

Attackers can:

- Replace a trusted app with a modified version
- Create a fake update that looks legitimate
- Inject hidden malicious behavior into normal apps

Even though the app looks normal, it may:

- Steal data
- Monitor activity
- Take control of the device

8. Conditions Required for Attack

This vulnerability does not work in all situations.

It requires:

- App must use v1 signature scheme
- Device must be Android 5.0 or above
- User must install the app from outside Google Play Store

So, users who install apps only from Play Store are generally safe.

9. Which Systems Are Affected

Affected:

- Apps signed with v1 scheme
- Devices running Android 5.0+

Not affected:

- Apps signed with v2 or higher
- Devices running Android 7.0+ with v2 signing enabled

Reason:
v2 signs the entire APK, so any modification breaks the signature.

10. Fix and Patches

- Google was informed on July 31, 2017
- Patch released in November 2017
- Public disclosure in December 2017 Android Security Bulletin
- Fully fixed in December 2017 security patch

Safe platforms:

- Google Play Store apps (protected by verification)
- F-Droid repository apps
- APKMirror (confirmed patched)

11. Key Takeaway

The vulnerability exists because v1 signature does not verify the entire file. Attackers exploit unused spaces in APK and DEX formats to inject malicious code while keeping the signature unchanged.

Solution:

- Always use v2 or higher signing
- Avoid installing apps from unknown sources
- Keep devices updated with latest security patches

3. Debuggable Flag

The `android:debuggable` attribute controls whether an app can be debugged.

If set to true:

- The app allows debugging tools to attach to it
- An attacker can assume the app's identity and permissions
- They can access private app data
- They can execute arbitrary code inside the app process

If set to false:

- Direct debugging is blocked
- An attacker would need root access to extract or manipulate data

Important development lifecycle point:

- During development, this is set to true for testing
- Before production release, it must be set to false

If developers forget to change it, it becomes a serious vulnerability

4. Debug Certificate and Signing (Full Detail)

When using Android Studio:

- The system automatically signs debug builds
- A debug keystore is created at `$HOME/.android/debug.keystore`
- Default passwords and keys are auto-managed

What a signing configuration includes:

- Keystore location
- Keystore password
- Key alias (name)
- Key password

Important limitation:

- Developers cannot directly edit the debug signing configuration
- But they can configure release signing manually

Security aspect:

- Debug certificates are insecure by design
- They are publicly known and not unique
- Because of this, app stores reject apps signed with debug certificates

Expiry detail:

- Debug certificate is valid for 30 years
- After expiry, build errors occur and a new certificate must be generated

5. Android Code Signing Certificate (Full Meaning)

Code signing is mandatory for publishing apps.

It ensures:

- Identity of the developer
- Integrity of the app (no tampering)
- Trust during updates (new versions must match original signature)

Without code signing:

- Apps cannot be installed or updated properly
- Google Play will reject the app

6. Browsable Activity (Complete Flow)

Browsable activity allows apps to respond to links clicked in a browser.

When a link is clicked, Android sends an intent with:

- action = android.intent.action.VIEW
- category = android.intent.category.BROWSABLE
- data = URL from the link

Examples:

Case 1:
Link → `tel:123456`
Result → Opens Dialer with number filled

Case 2:
Link → email format
Result → Opens email app with recipient filled

Case 3:
Link → `foo://my.test.com`
Default result → Error (browser doesn't understand it)

But if an app defines an IntentFilter for this:

- That app's activity will open
- It can handle that custom scheme

Key concept:

- Developers control app-link behavior using intent filters
- This is how deep linking works in Android

7. Exploiting Debuggable Android Applications (Full Attack Flow)

This section demonstrates a real exploitation process.

Goal: Change app behavior at runtime without modifying source code.

Step 1: Identify vulnerability

- Decompile APK using APKTool
- Check AndroidManifest.xml
- Look for `android:debuggable="true"`
  If present → app is exploitable

Important note: Only inspection is done, no code modification

Step 2: Setup environment

- Start emulator
- Install app
- Use command `adb jdwp`

What this does:

- Lists all running processes that allow debugging
- Each debuggable app exposes a unique port

JDWP meaning: Java Debug Wire Protocol

- Enables communication between debugger and app

Step 3: Identify target app

- Run `adb jdwp` before launching app
- Run again after launching app
- New port appears → this is target app's debug port

Step 4: Port forwarding

- Required because debugging is remote
- ADB forwards device port to local machine

Step 5: Attach debugger (JDB)

- Connect JDB to the target port
- Now attacker controls execution
- Jdb -attach localhost:<new-forwarded-port>

Step 6: Analyze app internals

Commands used:

- `classes` → lists all classes in app
- `methods <class>` → lists methods in a class

Step 7: Set breakpoint
Example:

- Target method: onClick
- Command: set breakpoint in that method
- *Stop in <method-name>*

Step 8: Trigger breakpoint

- User clicks button
- Execution pauses at breakpoint

Step 9: Inspect variables

- Use `locals` command
- View method arguments and variables

Step 10: Step through execution

- Use `next` to execute line by line
- Observe changes in variables

Step 11: Identify sensitive value

- Example string: “Try Again”

Step 12: Modify behavior

- Enter method using `step`
- Change variable using `set test = “Hacked”`
- Replace “Try Again” with “Hacked”

Step 13: Resume execution

- Use `run`
- App now shows modified output

Important understanding:

- No source code was changed
- Change happens during runtime

What attackers can do beyond this:

- Access sensitive variables
- Change app logic
- Inject malicious behavior
- Execute commands under app privileges
- Potentially gain shell access

Conclusion of exploitation:

- Debuggable apps are highly insecure in production
- This is a common and critical mistake checked in pentesting

8. AllowBackup Flag (Complete Explanation)

The `android:allowBackup` attribute controls whether app data can be backed up using ADB.

Default value:

- true

If true:

- Anyone with USB debugging enabled can extract app data
- Root access is NOT required
- Backup can include sensitive data like:
    - Passwords
    - Tokens
    - Financial details

If false:

- Backup via ADB is blocked
- Data extraction becomes much harder

Example:
`android:allowBackup="false"`

Important point:

- Developers must explicitly set it to false for sensitive apps
- Default setting can expose critical data if not changed

Final Complete Understanding

- Debuggable flag allows full runtime control of the app if enabled
- Debug certificates are auto-generated, insecure, and not allowed for production
- Code signing ensures authenticity and trusted updates
- Browsable activities control how links trigger app components
- Debuggable apps can be exploited using JDWP and JDB without modifying code
- AllowBackup flag can expose sensitive data if left enabled

This version now includes every detail from your content without skipping any concept.

1. Another way to intercept SSL traffic (Android)

Normally, tools like Burp Suite install a certificate on your device to intercept HTTPS traffic. But there's a deeper method:

- Android stores trusted certificates in a file called **`cacerts.bks`**
- Location: `/system/etc/security`

What attackers / pentesters do:

1. **Pull the certificate file**
    - Copy `cacerts.bks` from the device

2. **Modify it**
    - Use Java's **keytool** + **Bouncy Castle library** to add your own certificate (like Burp's)
3. **Push it back**
    - Replace the original file with the modified one
4. **Make system writable**
    - Android system partition is normally **read-only**
    - You must remount it as **read/write**
5. **For emulator (permanent change)**
    - Rebuild system image using `mks.yaffs2`
    - So changes don't disappear after reboot

Why this works:

You are basically telling Android:

> "Trust my fake certificate as if it's real"

So HTTPS traffic can now be intercepted.

2. Other tools for intercepting traffic

Besides Burp Suite, you can use:

- Charles Proxy
- mitmproxy

Key idea:

- They work similar to Burp
- But may offer:
    - Better UI (Charles)
    - More scripting/control (mitmproxy)

Example:

- Charles lets you directly download its certificate:
    - `charles.crt`
- Install it → start intercepting HTTPS traffic

3. Modifying server responses (real attack scenario)

Example situation:

- User tries to access restricted content

Server sends:
403 Forbidden

What a pentester does:

- Intercepts the response

Changes it to:
200 OK

Result:

- App thinks access is allowed
- User sees restricted content

Important insight:

This shows:

- The app **trusts server response blindly**
- No proper validation on client side

4. How apps try to prevent this (certificate pinning)

Secure apps use **certificate pinning**:

- App stores a **trusted certificate inside itself**
- When connecting:

It checks: Does the server certificate match the one inside the app?

If mismatch:

- Connection is blocked

Problem for pentesters:

- Your fake certificate (Burp/Charles) won't match
- So interception fails

What pentesters do next:

- Reverse engineer the app

- Find certificate validation logic
- Modify or bypass it
- Recompile the app

5. Extracting sensitive files from network traffic

Even if you don't modify traffic, you can **analyze captured packets**.

Tool used: Wireshark

6. How file extraction works (core idea)

When files are uploaded/downloaded:

They are sent as: multipart/form-data

This includes:

- File content
- File type headers

7. Steps to extract files in Wireshark

1. Open captured traffic file

Search for: multipart

2. Find a request (usually POST)
3. Right-click → **Follow TCP Stream**
4. Look at raw data:
    - Example:
        - `%PDF` → PDF file
        - `PNG` → Image
5. Save the data:
    - Choose **Raw format**
    - Save with correct extension (like `.pdf`)

Result:

You recover the exact file sent over the network

8. Alternative tool for file extraction

- NetworkMiner

Why use it:

- Easier than Wireshark
- GUI shows extracted files directly
- No manual effort needed

9. Big Picture (what you actually learned)

This entire topic revolves around:

A. Interception

- Breaking HTTPS protection using certificates

B. Manipulation

- Changing requests/responses (like 403 → 200)

C. Bypassing security

- Defeating certificate pinning via reverse engineering

D. Data extraction

- Pulling sensitive files from captured traffic

Android Application Security Vulnerability Assessment using QARK

QARK stands for **Quick Android Review Kit**. It is an open-source tool developed by LinkedIn for assessing the security of Android applications. QARK is used to identify common vulnerabilities in Android apps and provides guidance for remediation.

It helps penetration testers, developers, and security researchers perform **Android application security vulnerability assessments**.

QARK analyzes Android application source code or APK files to detect security weaknesses such as:

* Insecure exported components
* Weak cryptography
* Hardcoded credentials
* Insecure WebView usage
* Intent vulnerabilities
* Improper permission usage
* Tapjacking risks
* Debuggable applications
* Insecure storage issues

 # Purpose of QARK

✔ Detect Android app vulnerabilities early
✔ Improve secure coding practices
✔ Help developers fix issues
✔ Perform penetration testing
✔ Generate vulnerability reports

 # Input Supported by QARK

QARK can scan:

1. **APK files**
2. **Decompiled source code**
3. **AndroidManifest.xml**
4. Java source files

 # Installation of QARK

QARK runs mainly on Linux/Kali Linux.

Basic Installation:
git clone https://github.com/linkedin/qark.git
cd qark
pip install -r requirements.txt
python qark.py

 # Android Vulnerability Assessment Process using QARK

Step 1: Start QARK

Run: python qark.py

Step 2: Select Scan Type
Choose:
* Scan APK file
* Scan source code
* Scan project directory

Step 3: Provide Target Application
Example: /path/to/app.apk

Step 4: Automated Analysis
QARK begins scanning:
* Manifest file
* Activities
* Services
* Broadcast Receivers
* Content Providers
* Code logic
* WebView configurations
* Permissions

 # Vulnerabilities Detected by QARK
1. **Exported Components:** Checks whether Activities, Services, Receivers are exposed without protection.
Risk: Unauthorized access.

2. **Insecure Intents**
Detects implicit intent misuse or intent hijacking possibilities.

3. **Weak Permissions**
Checks dangerous permissions:
* READ_SMS
* READ_CONTACTS
* INTERNET

4. **Hardcoded Secrets**
Detects:
* API keys
* Passwords
* Tokens

5. **Insecure WebView**
Checks:

* JavaScript enabled unnecessarily
* addJavascriptInterface misuse
* Loading untrusted URLs

6. **Tapjacking Vulnerability**
Checks if UI overlay attacks are possible.

7. **Debuggable App**
If `android:debuggable="true"` is enabled.

8. I**nsecure Cryptography**
Detects use of:
* MD5
* SHA1
* ECB Mode

9. **Backup Enabled**
Checks: android:allowBackup="true"
Risk: App data extraction.

 # Output Report
QARK generates detailed HTML reports containing:
* Vulnerability name
* Severity
* Description
* Affected file
* Code snippet
* Remediation steps

 # Example Findings
Vulnerability	Risk
Exported Activity	Unauthorized access
Hardcoded API Key	Credential theft
Debuggable Enabled	Reverse engineering
WebView JS Enabled	XSS / Code execution
Weak Crypto	Data compromise

 # Advantages of QARK

✔ Free and open source
✔ Easy to use
✔ Detailed reports
✔ Good for developers

✔ Detects Android-specific flaws

 # Limitations of QARK
✖ Mostly static analysis
✖ Older project (less actively maintained)
✖ May miss runtime vulnerabilities
✖ False positives possible

 # QARK vs MobSF
Feature	QARK	MobSF
Static Analysis	Yes	Yes
Dynamic Analysis	No	Yes
Android Focused	Yes	Yes
Reporting	Good	Advanced
Ease of Use	Moderate	Easy GUI

 # Practical Use in Penetration Testing
QARK is used during:
1. APK review
2. Secure code audit
3. Pre-release security testing
4. Vulnerability discovery
5. Compliance checks

QARK is a powerful Android application vulnerability assessment tool that helps detect common Android security flaws such as insecure components, weak cryptography, hardcoded secrets, and WebView issues. It is valuable for Android app penetration testing and secure development.

QARK (Quick Android Review Kit) is an Android security assessment tool used to scan APKs and source code for vulnerabilities such as exported components, insecure intents, hardcoded credentials, weak crypto, and WebView issues. It generates reports with remediation guidance.

Introduction to Xposed Framework

Xposed Framework is a powerful Android modification framework that allows users and security researchers to **change the behavior of Android apps and the Android operating system without modifying APK files or flashing custom ROMs**.

It works by hooking into the Android runtime and letting modules intercept, modify, or replace methods of apps and system processes at runtime.

Xposed became popular for customization, debugging, reverse engineering, and security testing.

 # What is Xposed Framework?

Xposed is a **runtime hooking framework** for Android that enables modules to:
* Modify app behavior dynamically
* Change system UI features
* Bypass restrictions
* Hook methods in apps
* Customize Android features
* Analyze applications during runtime

It injects itself into the Android process startup mechanism and loads custom modules.

 # Key Concept of Xposed

Normally: App → Android Runtime → System

With Xposed: App → Xposed Hooks → Android Runtime → System

This means Xposed can intercept method calls before or after execution.

 # History of Xposed

* Developed by **rovo89**
* Became popular on Android 4.x to Android 8.x
* Later alternatives appeared:

1. **EdXposed**
2. **LSPosed**
3. **TaiChi** (partial support)

These support newer Android versions.

 # How Xposed Works

Xposed modifies the Android runtime (ART / Dalvik) and loads modules during boot.

It can:

* Hook Java methods
* Change method parameters

* Change return values
* Prevent functions from executing
* Run custom code before/after methods

 # Requirements
Traditionally:
* Rooted Android device
* Compatible Android version
* Custom recovery (sometimes)
Modern alternatives like **LSPosed** often use **Magisk**.

 # Components of Xposed
1. **Xposed Installer:** Used to install/manage framework and modules.
2. **Xposed Framework Core:** Hooks into Android runtime.
3. **Modules:** Apps that use Xposed APIs to modify behavior.

Examples:
* Privacy modules
* UI customization
* Root hiding
* Security testing modules

 # Method Hooking Example
Suppose app checks login: if(checkPassword(input))
Xposed module can hook: checkPassword()
And force return: true
Result: login bypass.

 # Common Uses of Xposed

1. UI Customization
* Change status bar
* Themes
* Navigation buttons

2. Privacy Control
* Fake location
* Block trackers
* Restrict permissions

3. Security Testing

* Bypass root detection
* Disable SSL pinning

* Modify authentication logic

4. App Modding

* Unlock hidden settings
* Remove ads
* Change behavior

 # Xposed in Android Security Testing
Used by penetration testers to:
* Hook security checks
* Inspect runtime values
* Intercept authentication
* Modify app logic
* Test insecure coding

Example Security Uses

Root Detection Bypass
Hook: isDeviceRooted()
Return: false

SSL Pinning Bypass
Hook certificate validation methods.

OTP Logic Testing
Intercept OTP verification methods.

Popular Xposed Variants
1. EdXposed: Supports newer Android versions using Riru.
2. LSPosed: Modern, stable, popular replacement.
3. TaiChi: No full root in some cases.

 # Advantages of Xposed

✔ No APK modification required
✔ Powerful runtime hooking
✔ Thousands of modules available
✔ Great for testing and customization
✔ Reversible changes

 # Limitations

✖ Usually requires root

✖ Can break apps/system updates
✖ Some banking apps detect it
✖ Security risk if malicious modules installed
✖ Older original Xposed not updated for latest Android

 # Xposed vs Frida

Feature	Xposed	Frida
Main Use	Permanent runtime hooks	Dynamic testing
Requires Scripts	No (modules)	Yes
Root Needed	Usually Yes	Often Yes
Best For	Device-wide changes	Pentesting/research
Ease	Easy after setup	More technical

 # Security Risks of Xposed

Because it can modify everything:

* Malicious modules may steal data
* Apps may be tampered
* Weakens device trust model
* Banking apps may block rooted/Xposed devices

Xposed Framework is a powerful Android runtime modification platform that enables apps and system functions to be changed dynamically through method hooking. It is widely used for customization, reverse engineering, and mobile security testing. Modern replacements such as LSPosed continue its legacy.

Xposed Framework is an Android runtime hooking framework that allows modification of apps and system behavior without changing APK files. It uses modules to intercept methods for customization, security testing, privacy control, and reverse engineering.

No.	Static Analysis	Dynamic Analysis
1	Analysis of application **without executing the code**	Analysis of application **while executing the code**
2	Performed on **source code, bytecode, or binaries**	Performed on **running application in real environment**

3	Used to find **coding errors and vulnerabilities early**	Used to find **runtime issues and behavioral flaws**
4	Does not require app installation	Requires app to be installed and executed
5	Faster to perform	Slower due to execution and monitoring
6	Cannot detect runtime issues like memory leaks easily	Can detect runtime issues like memory leaks, crashes
7	Provides full code coverage (entire code can be scanned)	Limited to executed paths only
8	Easier to automate using tools	Harder to automate completely
9	No risk of harming system (safe analysis)	May pose risk if malware is executed
10	Helps in understanding code structure and logic	Helps in understanding real-time behavior
11	Cannot detect dynamic behaviors (e.g., runtime API calls)	Can detect dynamic behaviors and interactions
12	Used in early stages of development	Used in testing and post-development stages
13	Examples: Code review, decompilation, scanning	Examples: Debugging, monitoring, runtime tracing
14	Tools: JADX, SonarQube	Tools: Frida, Burp Suite
15	Cannot observe actual user interaction	Can observe real user interaction and system response

Diva Vulnerabilities

Based on the provided technical materials, here is a detailed breakdown of the vulnerabilities in the DIVA (Damn Insecure Vulnerable Application) for your exam preparation.

1. Insecure Logging

- **Why it is a vulnerability**: Android applications often generate logs for debugging. If sensitive data (like passwords or credit card numbers) is sent to the log, it can be read by any application with `READ_LOGS` permission (on older versions) or by anyone with physical access to the device via ADB.
- **How it is caused**: Developers use the `android.util.Log` class (e.g., `Log.d`, `Log.i`, `Log.v`) to print sensitive variables directly into the system log.
- **How attackers exploit it**: Attackers use the `adb logcat` command to view real-time logs and filter for the application's process ID or specific keywords like "diva" to extract sensitive information.
- **How to safeguard**:
    - Avoid logging sensitive information in production code.
    - Use ProGuard or similar tools to automatically remove log statements during the build process for release versions.

2. Hardcoding Issues

- **Why it is a vulnerability**: Hardcoding secrets makes them permanent fixtures of the application's binary. Since Android APKs can be easily decompiled, these secrets are not truly hidden.
- **How it is caused**: A developer explicitly defines a constant value (like an API key, "vendersecretkey", or admin password) directly within the Java source code.
- **How attackers exploit it**: Attackers use decompiler tools like **JD-GUI** or **JADX** to view the source code and search for string comparisons (e.g., `.equals()`) or variable assignments that contain sensitive keys.
- **How to safeguard**:
    - Never store sensitive credentials in the source code.
    - Perform authentication and sensitive comparisons on the server-side rather than the client-side.

3. Insecure Data Storage

This category involves how an app saves persistent data. If not secured, this data can be accessed by unauthorized parties.

Subtype	**How it is Caused**	**Exploitation Method**

Shared Preferences	Data is saved in key-value pairs using `SharedPreferences` without encryption.	Attackers navigate to `/data/data/jakhar.aseem.diva/shared_prefs` and `cat` the XML files to read plaintext data.
SQLite Database	Sensitive data is inserted into local SQLite tables in plaintext.	Attackers access `/data/data/jakhar.aseem.diva/databases/`, open the database with `sqlite3`, and run `SELECT` queries.
Temporary Files	The app creates temporary files (e.g., using `createTempFile`) containing credentials and fails to delete them.	Attackers search the app's internal directory for files with `.tmp` extensions and read their contents.
External Storage (SD Card)	Data is written to the SD card (`/sdcard/`), which is often globally readable.	Attackers use `ls -a` to find hidden files (like `.uinfo.txt`) on the SD card that contain sensitive information.

- **How to safeguard**:
    - Use **EncryptedSharedPreferences** or the **Android Keystore** system to encrypt data before storage.
    - Always use `MODE_PRIVATE` for internal files.
    - Avoid using external storage for sensitive data, as it may be world-readable.

4. Input Validation & Data Sanitization (IDS)

- **Why it is a vulnerability**: Failure to validate user-provided data allows attackers to craft inputs that change the application's intended logic.
- **Subtypes**:

    - **SQL Injection (SQLi)**: Caused by concatenating user input directly into SQL queries (e.g., `SELECT * FROM table WHERE user = ' + input + '`). Attackers use payloads like `' or '1'='1` to bypass authentication.
    - **Abuse Web View (LFI)**: Caused when a `WebView` loads a URL from user input without validation. Attackers can use the `file://` scheme (e.g., `file:///sdcard/demo.txt`) to read local system files instead of web pages.
- **How to safeguard**:
    - **SQLi**: Use **Parameterized Queries** or Prepared Statements to separate code from data.
    - **WebView**: Sanitize URLs to ensure they use allowed schemes (like `https://`) and disable local file access (`setAllowFileAccess(false)`).

5. Access Control Issues

- **Why it is a vulnerability**: Occurs when an application fails to properly authenticate or authorize a user before granting access to a protected resource.
- **How it is caused**: Developers may leave sensitive Activities "exported" in the `AndroidManifest.xml` without requiring permissions or checking if the caller is authorized.
- **How attackers exploit it**: Attackers use the **Activity Manager (am)** tool via ADB to start private activities directly (e.g., `am start -n package/activity`), bypassing the intended UI flow or PIN screens.
- **How to safeguard**:
    - Set `android:exported="false"` for activities that should not be accessible by other apps.
    - Implement robust session management and re-authenticate users before displaying sensitive data.

Access Control Issues - I (Insecure Activities)

- **Why it is a vulnerability:** This arises when an application does not authenticate a user or fails to perform an authorization check before granting access to a protected resource. An attacker with insufficient privileges can bypass the intended user interface to see sensitive information.
- **How it is caused:** Developers may leave sensitive components, like Activities, "exported" in the AndroidManifest.xml without proper permission requirements. If a component is exported, it can be accessed by other applications on the same device.
- **Subtypes:**
    - **Insecure Intent Calling:** Using intents to call activities that disclose secret data or threaten user privacy.
- **How attackers exploit them:** * **Activity Manager (am):** Attackers use the command adb shell am start -a [intent_action] to force-start an activity directly from the command line, bypassing the app's internal buttons and logic.

    - **Drozer:** Security frameworks like Drozer are used to identify the "Attack Surface," specifically looking for exported activities that can be launched externally.
- **How to safeguard:**
    - **Set Exported to False:** Ensure android:exported="false" is set for all activities that do not need to be accessed by other apps.
    - **Declare Permissions:** If a component must be shared, the host application should declare and require specific permissions for the caller.

2. Authentication-Based Access Control Issues

- **Why it is a vulnerability:** This occurs when an attacker can authenticate themselves but still bypass specific authorization mechanisms (like a PIN or password check) to reach protected resources. It effectively abuses the mechanism meant to protect sensitive credentials.
- **How it is caused:** * **Logic Flaws:** The application might rely on a simple boolean check (e.g., check_pin) passed through an intent to determine if a user has authenticated.
    - **Exported Content Providers:** Sensitive data repositories are sometimes left exported, allowing direct database queries that bypass the authentication UI.
- **Subtypes:**
    - **Intent Filter Bypass:** Bypassing a PIN screen by sending a specific "extra" boolean value in an intent.
    - **Insecure Content Providers:** Directly querying data providers (like a notes database) that lack access controls.
- **How attackers exploit them:**
    - **Bypassing PINs with Intents:** Using ADB to send an intent with an extra boolean flag set to false: am start -a [action] --ez "check_pin" false. This tricks the activity into opening without a valid PIN check.
    - **Drozer Provider Queries:** Attackers use Drozer to find accessible URIs (e.g., content://.../notes/) and query them directly to dump the entire database contents without ever knowing the PIN.
- **How to safeguard:**
    - **Robust Session Management:** Do not rely on client-side intent extras for authentication state.
    - **Secure Content Providers:** Use the android:permission attribute in the manifest to restrict access to content providers to only authorized callers.
    - **Database Encryption:** Ensure sensitive data stored within providers is encrypted so that even a direct query reveals only ciphertext.